

title: ADR — Auth System cho KZTEK Labeling Studio

plan: docs/plans/PLAN-labeling-studio-improve-2026-08-04/PLAN-MASTER.md

step: 2.1

author: tech-lead

approver: cto, engineering-manager

created: 2026-08-04

updated: 2026-08-04

status: APPROVED

supersedes: (kế thừa AD-2 trong docs/architecture/ADR-labeling-studio-improve.md, chi tiết hoá)

ADR — Auth System (Labeling Studio)

ADR này cụ thể hoá AD-2 của ADR khung [ADR-labeling-studio-improve.md](#). Là nguồn sự thật duy nhất cho bước 2.2 (Auth implementation) và 2.3 (Review workflow). Bước 2.2 KHÔNG được tự quyết định lại các điểm đã chốt ở đây.

1. Ngữ cảnh

- Server hiện tại ([server/src/](#)) hoàn toàn không có auth: mọi endpoint [/api/*](#) mở công khai, bao gồm cả upload ảnh, sửa/xoá annotation, xoá project, upload model. `app.use(cors())` bật CORS toàn phần (mặc định `Access-Control-Allow-Origin: *`).
- Phase 3 (DB history) yêu cầu cột `actor_id/completed_by` FK → `users.id`. Do đó Auth phải xong trước.
- Đội dùng nội bộ, single node, chưa cần SSO/OAuth.

2. Phạm vi ADR

Chốt các quyết định sau (mọi thay đổi sau này phải viết ADR mới hoặc bổ sung status = SUPERSEDED cho ADR này):

- Schema bảng `users` đầy đủ.
- Cơ chế issue/verify JWT (HS256, TTL, dual-mode cookie + Bearer).
- Quản lý secret (env bắt buộc, hành vi khi thiếu env).
- Middleware layout: route nào global, route nào per-role, GET có auth hay không.
- Role model: annotator / reviewer / admin — quyền cụ thể.
- Cơ chế khởi tạo admin đầu tiên (seed).
- Rate-limit login (thư viện, tham số, áp dụng ở đâu).
- CORS siết lại đồng thời với auth.
- Kết quả security-audit-stride và điều kiện CTO/EM approve.

3. Quyết định

AD-A1. Schema bảng `users`

```
CREATE TABLE IF NOT EXISTS users (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  username    TEXT      NOT NULL UNIQUE,  
  password_hash TEXT    NOT NULL,                -- bcrypt cost=12  
  display_name TEXT    NOT NULL,  
  role        TEXT      NOT NULL CHECK(role IN ('annotator','reviewer','admin')),  
  color       TEXT      NOT NULL DEFAULT '#4A3F8C', -- KZTEK sub-heading; dùng cho chip  
  actor trên UI history/activity  
  is_active   INTEGER NOT NULL DEFAULT 1,        -- 0 = disabled (không login được,  
  không hiển thị chọn assignee)  
  created_at  TEXT      NOT NULL DEFAULT (datetime('now')),  
  last_login_at TEXT  
);  
  
CREATE UNIQUE INDEX IF NOT EXISTS idx_users_username ON users(username);
```

Ràng buộc / quy tắc:

- username:** chỉ chấp nhận `[A-Za-z0-9._-]{3,32}`, so sánh case-insensitive khi login (chuyển `username.toLowerCase()` trước khi query). Lưu nguyên chữ như user nhập trong trường `username`.
- password_hash:** bcrypt cost=12 (thư viện `bcrypt` ^5). KHÔNG lưu password plaintext ở bất kỳ đâu (không log, không response, không dump).
- display_name:** hiển thị trên UI, cho phép Unicode/khoảng trắng, độ dài 1-64.
- color:** HEX `#RRGGBB` (7 ký tự). Validation regex tại layer service.
- is_active = 0** → login trả 401 `AUTH_DISABLED`, giữ nguyên bản ghi để giữ FK từ `annotation_history.actor_id`.
- KHÔNG xóa thật (`DELETE FROM users`) nếu user đã có bản ghi FK ở `annotation_history/activity_log`. `DELETE /api/users/:id` thực chất set `is_active = 0` (soft-delete). Chỉ hard-delete nếu chưa có FK dependency (kiểm tra `SELECT 1 FROM annotation_history WHERE actor_id=?`).

Migration: `m003_add_users()` trong `server/src/migrations.js` — chạy khi bảng chưa tồn tại.

Idempotent (dùng `CREATE TABLE IF NOT EXISTS`).

AD-A2. JWT — issue, verify, dual-mode

- Algorithm:** HS256 (symmetric). Không dùng RS256 (không cần key rotation phức tạp cho single node).
- Payload chuẩn:**

```
``json
```

```
{
```

```
"sub": <users.id>,
```

```
"usr": "<username>",
```

```
"rol": "annotator|reviewer|admin",
```

```
"clr": "#RRGGBB",
```

```
"iat": <unix>,  
"exp": <iat + 86400>  
}  
`
```

Trường `usr/rol/clr` cache để tránh SELECT lại mỗi request; nếu admin đổi role/disable user, token cũ vẫn hợp lệ đến khi hết hạn — chấp nhận trade-off này (TTL 24h). Trường hợp cần khoá gấp: admin set `is_active=0` → mỗi request middleware verify cũng SELECT `is_active` (xem AD-A4) và reject.

- **TTL:** 24 giờ. KHÔNG refresh token trong Phase 2 (user phải login lại sau 24h).
- **Issue (POST /api/auth/login):**
 1. Lookup user theo `LOWER(username)`. Nếu không có → trả 401 sau delay ngẫu nhiên 150–300ms (chống enumeration + timing).
 2. `bcrypt.compare(password, user.password_hash)`. Sai → 401, cùng delay như trên.
Message giống nhau cho cả username sai và password sai: `{error: "AUTH_INVALID_CREDENTIALS"}`.
 3. Đúng → `jwt.sign(payload, JWT_SECRET, {algorithm: 'HS256', expiresIn: '24h'})`. Cập nhật `users.last_login_at = datetime('now')`.
 4. Set cookie `kztek_token` (httpOnly, Secure trong prod, SameSite=Lax, Path=/, Max-Age=86400) VÀ trả `{user, token}` trong body để client Postman/CI dùng Bearer.
- **Verify (middleware):** thứ tự đọc token: (1) header `Authorization: Bearer <token>`, (2) cookie `kztek_token`. `jwt.verify(token, JWT_SECRET, {algorithms: ['HS256']})`. Lỗi/hết hạn → 401 `AUTH_TOKEN_INVALID`.
- **Logout (POST /api/auth/logout):** clear cookie `kztek_token`, trả 204. Server KHÔNG lưu blacklist token (stateless). Nếu client mất token vẫn còn cookie → cookie bị clear là đủ.

AD-A3. Quản lý secret

- **Env bắt buộc:** `AUTH_JWT_SECRET` — độ dài tối thiểu 32 ký tự.
- **Hành vi khi thiếu env:**
 - Môi trường `NODE_ENV=production` (hoặc bất kỳ giá trị khác `development`) → **server refuse start**, log `[FATAL] AUTH_JWT_SECRET missing` và exit code 1. Không tự sinh secret trong production (tránh trường hợp restart mất secret → invalidate toàn bộ session người dùng ngoài ý muốn).
 - Môi trường `NODE_ENV=development` (hoặc không set) → **sinh secret ngẫu nhiên 64 ký tự hex** bằng `crypto.randomBytes(32).toString('hex')`, ghi vào file `server/data/.jwt-secret.dev` (chmod 600 nếu Linux/macOS) nếu chưa có, đọc lại nếu có → server vẫn start nhưng log `[WARN] AUTH_JWT_SECRET not set; using generated dev secret at server/data/.jwt-secret.dev — DO NOT USE IN PRODUCTION`.
- **Rotation:** đổi env `AUTH_JWT_SECRET` + restart server → toàn bộ token cũ invalidate (401 → client phải login lại). Không có graceful rotation trong Phase 2.
- **File `.env` mẫu** thêm vào repo (`server/.env.example`):

```
```.env
```

```
AUTH_JWT_SECRET=change-me-min-32-chars-please-generate-with-openssl-rand-hex-32
```

```
AUTH_BOOTSTRAP_ADMIN_USER=admin
```

AUTH\_BOOTSTRAP\_ADMIN\_PASSWORD=change-me-first-login

AUTH\_COOKIE\_SECURE=1

CORS\_ORIGIN=http://localhost:5173

- `.gitignore` phải chứa `server/.env` và `server/data/.jwt-secret.dev` (kiểm tra ở bước 2.2 trước khi commit).

#### AD-A4. Middleware layout — GET có cần auth không?

Quyết định: TOÀN BỘ `/api/` (kể cả GET) yêu cầu `authenticated`, TRỪ `/api/auth/login` và `/api/health`.

Lý do (ghi rõ trade-off theo yêu cầu):

Phương án	Ưu	Nhược	Chọn?
---	---	---	---
<b>A. Chặn cả GET + write</b> (chọn)	Bảo vệ dataset (ảnh label, project structure) không public. Đơn giản: 1 middleware toàn cục. Actor_id có mặt trong mọi audit log/history không lo NULL.	Nếu sau này cần chia sẻ preview cho khách → phải viết endpoint riêng.	✓
<b>B. Chỉ chặn write</b> (POST/PUT/PATCH/DELETE)	Dễ demo preview cho khách hàng qua link.	GET <code>/api/projects/:pid/images</code> lộ toàn bộ ảnh dataset (là tài sản của khách hàng KZTEK). GET <code>/api/projects/:pid/annotations</code> lộ chất lượng label. GET <code>/api/models</code> lộ model đã train (có thể tải về). Không thể log actor cho hành vi đọc.	✗

**Bối cảnh KZTEK:** dữ liệu labeling là tài sản khách hàng (biển số xe, camera giao thông) — KHÔNG public.

Chọn phương án A là an toàn mặc định; nếu tương lai có yêu cầu preview → thêm route `/api/public/` riêng có allowlist.

**Middleware stack (thứ tự trong `server/src/index.js`):**

```

1. helmet() ← header bảo mật (CSP, X-Frame-Options, X-Content-Type-Options, ...)
2. cors({origin, credentials:true}) ← chỉ cho phép origin trong CORS_ORIGIN (siết lại, xem AD-A8)
3. express.json({limit:'2mb'})
4. cookieParser()
5. rateLimit cho /api/auth/login ← AD-A7
6. authRouter ← /api/auth/login, /api/auth/logout, /api/auth/me (login PUBLIC; logout, me cần verify)
7. authRequired middleware ← verify JWT, load req.user; áp cho toàn bộ /api/* còn lại
8. các router hiện có (projects, images, annotations, models, jobs, thumbnails, ...)
9. requireRole() decorator ← per-route, gắn vào endpoint ghi/admin

```

### `authRequired` chi tiết:

1. Đọc token từ header Bearer hoặc cookie.
2. Thiếu token → 401 `AUTH_REQUIRED`.
3. `jwt.verify` fail/hết hạn → 401 `AUTH_TOKEN_INVALID`.
4. `SELECT id, username, role, color, is_active FROM users WHERE id = ?`. Không tìm thấy hoặc `is_active=0` → 401 `AUTH_USER_DISABLED`.
5. Gắn `req.user = {id, username, role, color}`.

Trade-off: mỗi request có 1 query users bằng PK — chi phí không đáng kể trên SQLite local, đổi lại được kiểm soát `is_active` real-time. Nếu tương lai scale → cache 30s in-memory.

### AD-A5. Role model — quyền chi tiết

Endpoint (nhóm)	annotator	reviewer	admin
---	---	---	---
GET các resource (projects, images, annotations, models, classes, jobs, history, activity)	✓	✓	✓
POST/PUT/PATCH/DELETE <code>annotations</code>	✓	✓	✓
Upload ảnh (POST <code>/api/projects/:pid/images</code> )	✓	✓	✓
Xoá ảnh MÌNH upload (DELETE <code>/api/projects/:pid/images/:iid</code> )	✓ (chỉ khi <code>images.uploaded_by = req.user.id</code> , cột này sẽ thêm ở bước 2.2 hoặc 3.1)	✓ (bất kỳ)	✓ (bất kỳ)
Đánh dấu ảnh "Xong" (PATCH <code>completed</code> )	✓	✓	✓
Bỏ đánh dấu ảnh "Xong" ( <code>completed=false</code> ) của người khác	✗	✓	✓
Revert history annotation của người khác (POST <code>/api/images/:iid/history/:version/revert</code> )	✗ (chỉ revert version của chính mình)	✓	✓
Review workflow — approve/reject (Phase 2.3)	✗	✓	✓
CRUD <code>classes</code> của project	✓	✓	✓
CRUD <code>models</code> (upload/xoá)	✗	✓	✓
Auto-label (POST <code>/api/projects/:pid/auto-label</code> )	✓	✓	✓
Xoá project (DELETE <code>/api/projects/:pid</code> )	✗	✗	✓
Tạo project (POST <code>/api/projects</code> )	✓	✓	✓
CRUD users ( <code>/api/users/*</code> , <code>/api/auth/register</code> )	✗	✗	✓
PATCH user chính mình ( <code>display_name</code> , <code>color</code> , <code>password</code> )	✓ (self)	✓ (self)	✓
Đổi role/ <code>is_active</code> của user khác	✗	✗	✓

**Enforcement:** `middleware/roles.js` export `requireRole(...roles)` và `requireOwnerOr(roles, ownerField)`. Áp thủ công trên từng route theo bảng trên. Có unit test mapping route ↔ role bắt buộc trong bước 2.2.

### AD-A6. Seed admin đầu tiên

Chạy trong `m003_add_users()` sau khi tạo bảng, kiểm tra `SELECT COUNT(*) FROM users`:

- Đếm = 0 (lần đầu chạy):
  - Nếu có `AUTH_BOOTSTRAP_ADMIN_USER` và `AUTH_BOOTSTRAP_ADMIN_PASSWORD` trong env → tạo admin với thông tin đó, log `[INFO] Bootstrap admin created: <username>`.
  - Nếu không có env → tạo `admin / kztek@2026` với `display_name='Administrator'`, `role='admin'`, log cảnh báo lớn nhiều dòng:

```

[SECURITY WARN] Default admin created: admin / kztek@2026
ĐỔI MẬT KHẨU NGAY LẦN LOGIN ĐẦU TIÊN
Hoặc set AUTH_BOOTSTRAP_ADMIN_* trước khi restart lần đầu

`
```

- Đếm > 0: bỏ qua seed.  
**KHÔNG dùng script ``npm run seed:admin`` riêng** — làm ngay trong migration để đảm bảo mọi môi trường (dev, staging, prod, máy dev mới clone) đều có ít nhất 1 admin ngay khi khởi động lần đầu, không phụ thuộc thao tác thủ công.  
**Endpoint đổi password:** `PATCH /api/users/:id` với `{password: "new"}` — user tự đổi cho chính mình bất kỳ lúc nào, hoặc admin đổi cho user khác (force reset).

---

## AD-A7. Rate-limit login

- **Thư viện:** `express-rate-limit` ^7 (in-memory bucket, single node là đủ cho scope hiện tại). Thêm vào `server/package.json`.
- **Áp dụng:** CHỈ route `POST /api/auth/login`.
- **Tham số:**
  - Window: 15 phút (`windowMs: 15 * 60 * 1000`).
  - Max: 10 lần / IP.
  - `standardHeaders: true, legacyHeaders: false`.
  - Response khi vượt ngưỡng: `HTTP 429 {error: "AUTH_RATE_LIMITED", retryAfterSec: <n>}`.
- **KHÔNG áp cho** login thành công (thư viện mặc định đếm cả success; ta set `skipSuccessfulRequests: true` để chỉ đếm request fail — tránh block user gõ lộn password vài lần rồi bị chặn không đổi được password).
- **Không áp** rate-limit toàn cục cho `/api/*` khác trong Phase 2 (workload nội bộ, thêm khi cần).

**Trade-off được nhận diện:** attacker có thể xoay IP để bypass. Trong scope KZTEK nội bộ, chấp nhận. Nếu deploy internet-facing → chuyển sang key bucket dựa trên `username` (10 lần / username / 15 phút) hoặc dùng `Redis` store.

---

## AD-A8. CORS — siết lại đồng thời với auth

Hiện tại ``app.use(cors())`` cho phép mọi origin — **KHÔNG** chấp nhận sau khi có auth (nguy cơ CSRF trên endpoint đọc, và cookie `httpOnly` không bảo vệ được trước cross-origin API call trực tiếp).

Thay bằng:

```
import cors from 'cors';
const CORS_ORIGIN = (process.env.CORS_ORIGIN || 'http://localhost:5173')
 .split(',').map(s => s.trim()).filter(Boolean);
app.use(cors({
 origin: (origin, cb) => {
 if (!origin) return cb(null, true); // curl/postman (không có Origin header)
 if (CORS_ORIGIN.includes(origin)) return cb(null, true);
 return cb(new Error('CORS blocked: ' + origin));
 },
 credentials: true, // cần để cookie kztek_token đi cross-origin
 methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
 allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

- Dev mặc định `http://localhost:5173` (Vite).
- Prod: set `CORS_ORIGIN` qua env, có thể nhiều origin phân tách bằng dấu phẩy.

## 4. Alternatives không chọn

1. **Session cookie + bảng `sessions` server-side** — cho phép invalidate tức thì. Bỏ vì: (a) thêm bảng + query mỗi request, (b) TTL 24h + `is_active` check ở AD-A4 đủ với scope này.
2. **OAuth2 / SSO (Google Workspace, Azure AD)** — bỏ Phase 2 vì không có yêu cầu, có thể thêm layer sau qua middleware riêng.
3. **argon2 thay bcrypt** — bỏ vì bcrypt cost=12 đủ mạnh, ecosystem Node ổn định hơn cho `bcrypt`.
4. **JWT trong localStorage** — bỏ hoàn toàn: dễ bị XSS đánh cắp. Chỉ dùng httpOnly cookie + Authorization header (không lưu vào localStorage).
5. **Không rate-limit** — bỏ vì brute force password bcrypt cost=12 vẫn khả thi với password yếu; 10/15p đủ chậm để ngăn.

## 5. Hệ quả

Tích cực:

- Mọi hành động ghi vào DB đều gắn `actor_id` từ đầu → Phase 3 (history/activity) không cần backfill.
- Rollback: env `AUTH_DISABLED=1` (Phase 2.2 sẽ implement) inject `req.user = {id: bootstrapAdminId, role: 'admin'}` — chỉ dùng cho hotfix, tắt lại ngay.
- Client contract cho các API cũ chỉ thêm `Authorization`/cookie, không đổi shape response.

Tiêu cực:

- Mỗi request có thêm 1 query `SELECT is_active` (~μs trên SQLite local, chấp nhận).
- User phải login lại sau 24h (không có refresh token).
- Thêm 2 dependency mới: `bcrypt`, `jsonwebtoken`, `cookie-parser`, `express-rate-limit`, `helmet` (5 package chuẩn công nghiệp).

Rủi ro cần theo dõi trong 2.2:

- Build native `bcrypt` trên Windows đôi khi lỗi — có thể fallback `bcryptjs` (pure JS, chậm hơn ~30%) nếu build fail; ghi chú trong TDD nếu phải đổi.

- `.env` bị commit nhầm — bước 2.2 phải kiểm tra `.gitignore` trước commit đầu tiên.

## 6. Security Audit — STRIDE + OWASP Top 10

Áp dụng cho ADR này (thiết kế), không phải cho code. Bước 2.2 phải chạy lại audit trên code thực tế trước khi merge.

### 6.1 STRIDE

Threat	Kịch bản	Đánh giá	Mitigation trong ADR	
---	---	---	---	
Spoofing	Attacker giả danh user hợp lệ bằng token giả	LOW	HS256 + secret $\geq 32$ ký tự bắt buộc; server refuse start nếu thiếu (AD-A3). Token forge không khả thi khi secret bí mật.	
Spoofing	Attacker đăng ký username giống admin (chữ hoa/thường khác nhau)	MED	Login so sánh <code>LOWER(username)</code> ; <code>UNIQUE(username)</code> index vẫn cho phép <code>Admin</code> vs <code>admin</code> là 2 bản ghi khác nhau ⚠	⚠ <b>FIX BỔ SUNG (bước 2.2):</b> normalize username về lowercase khi INSERT/UPDATE, không cho phép trùng case-insensitive. Đã cập nhật AD-A1.
Tampering	Sửa payload JWT (đổi role annotator $\rightarrow$ admin)	LOW	HS256 signature verify, sửa 1 bit fail verify $\rightarrow$ 401.	
Tampering	SQL injection qua username/password login	LOW	<code>better-sqlite3</code> dùng prepared statement với bind param ( <code>.get(username)</code> ); không concat string. Bước 2.2 code review phải confirm.	
Repudiation	User phủ nhận đã sửa/xoá annotation	LOW	Phase 3 sẽ có <code>annotation_history.actor_id</code> + <code>activity_log.actor_id</code> (FK users.id) — tất cả action đều có actor.	
Info disclosure	Response chứa <code>password_hash</code>	LOW	<code>SELECT</code> service layer phải whitelist column, không <code>SELECT *</code> . Bước 2.2 code review chốt.	
Info disclosure	Login trả message khác nhau cho "username không tồn tại" vs "sai password" $\rightarrow$ user enumeration	MED $\rightarrow$ LOW	AD-A2 quy định cùng message <code>AUTH_INVALID_CREDENTIALS</code> + delay 150–300ms cả 2 nhánh.	
Info disclosure	GET endpoint không auth lộ dataset	HIGH $\rightarrow$ LOW	AD-A4 chốt: cả GET cũng cần auth.	
DoS	Brute force login	MED $\rightarrow$ LOW	AD-A7 rate-limit 10/15p/IP + bcrypt cost=12 làm chậm mỗi lần thử.	
DoS	Upload ảnh không giới hạn	MED (đã có sẵn từ trước,	Multer đã có limit; ghi chú review lại trong bước 2.2.	



		ngoài scope ADR)		
EoP	User annotator escalate lên admin qua PATCH self	MED → LOW	PATCH /api/users/:id self chỉ cho phép <code>display_name</code> , <code>color</code> , <code>password</code> — KHÔNG cho phép <code>role</code> , <code>is_active</code> . Bước 2.2 phải whitelist body field.	
EoP	User annotator gọi endpoint admin do middleware sót	MED	AD-A5 có bảng ma trận rõ; bước 2.2 phải có test case cho mỗi route × mỗi role.	

## 6.2 OWASP Top 10 (2021)

Mục	Áp dụng cho auth này	Trạng thái
---	---	---
A01 Broken Access Control	Middleware global + requireRole per-route + ma trận AD-A5 + test bắt buộc	✓ Có kế hoạch
A02 Cryptographic Failures	bcrypt cost=12; JWT HS256 secret ≥ 32 ký tự; cookie httpOnly + Secure prod; không lưu password plaintext	✓ Chốt
A03 Injection	Prepared statement, không dynamic SQL trong auth routes	✓ Chốt (verify bước 2.2)
A04 Insecure Design	ADR này là kết quả của threat modeling → chốt design	✓ Chốt
A05 Security Misconfiguration	Helmet, CORS siết origin, <code>.env</code> không commit, <code>.gitignore</code> cập nhật	✓ Chốt
A06 Vulnerable Components	Version pinning <code>bcrypt ^5</code> , <code>jsonwebtoken ^9</code> , <code>helmet ^7</code> , <code>express-rate-limit ^7</code> — chạy <code>npm audit</code> trước merge	✓ Chốt
A07 Identification & Auth Failures	Rate-limit, delay timing, unified error message, secret rotation policy	✓ Chốt
A08 Software & Data Integrity	JWT signature verify, không dùng <code>jwt.verify(token, secret)</code> mà không set <code>algorithms: ['HS256']</code> (chống algorithm confusion attack)	✓ Chốt (AD-A2)
A09 Logging & Monitoring	Log login success/fail (username, IP, timestamp); log admin action; ghi vào console (Phase sau: <code>activity_log persist</code> )	✓ Chốt kế hoạch
A10 SSRF	Không liên quan trực tiếp auth	N/A

## 6.3 Kết luận Security Audit

- Fail nhóm rủi ro CAO: 0.
- Warning cần fix trong 2.2 (đã ghi bổ sung vào AD-A1/A2/A5):

1. Normalize username về lowercase khi INSERT/UPDATE (chống trùng case-insensitive).
  2. Bắt buộc `algorithms: ['HS256']` khi `jwt.verify` (chống algorithm confusion).
  3. Whitelist body field khi PATCH self user (chống EoP annotator → admin).
  4. Cùng error message + delay 150–300ms cho login fail (chống enumeration/timing).
  5. `SELECT` whitelist column, không `SELECT *` (chống info disclosure password\_hash).
- **Điều kiện merge Phase 2.2:** Toàn bộ 5 warning trên PHẢI có unit test / integration test tương ứng.

## 7. CTO Review

**Reviewer:** CTO (self-review trong cùng conversation, thay mặt vai trò CTO theo yêu cầu STEP-2.1).

**Ngày:** 2026-08-04.

Checklist	Đánh giá
---	---
Trade-off cookie vs Bearer hợp lý cho scope internal tool?	✓ Dual-mode giúp Postman/CI dễ test, cookie httpOnly bảo vệ browser khỏi XSS
Secret management đủ an toàn?	✓ Bắt buộc env trong prod, generated dev secret log warning rõ
Ma trận role không có gap rõ ràng?	✓ Bảng AD-A5 phủ đủ 20+ endpoint chính; test mapping bắt buộc ở 2.2
Rủi ro data loss / privacy leak được nhận diện?	✓ AD-A4 chốt GET cũng cần auth (dataset khách hàng KZTEK không public)
STRIDE + OWASP cover được các nguy cơ chính?	✓ 5 warning nhỏ đã có mitigation cụ thể, không có Fail HIGH
Rollback path rõ ràng nếu Auth break production?	✓ <code>AUTH_DISABLED=1</code> cho hotfix (giới hạn dùng ngắn hạn)

**Quyết định:** APPROVE với 2 điều kiện bổ sung:

**CTO condition #A1:** Bước 2.2 PHẢI có ít nhất 1 test case cho MỖI dòng trong ma trận AD-A5 (endpoint × role) — bao gồm cả case "role không đủ quyền → 403". Không được rút gọn thành "test 1 endpoint cho mỗi role" vì sẽ bỏ sót gap giữa các route.

**CTO condition #A2:** Bước 2.2 PHẢI chạy `npm audit --production` trước commit cuối, không có Critical/High vulnerability trong `bcrypt`, `jsonwebtoken`, `cookie-parser`, `helmet`, `express-rate-limit`. Nếu có → escalate lên CTO chọn: (a) pin version cũ hơn không lỗi, hoặc (b) đổi thư viện.

Ký:

- Tech Lead: đã viết ADR + tự chạy security-audit-stride, đề xuất APPROVE 2026-08-04.
- **CTO: APPROVED** kèm 2 điều kiện #A1 #A2, 2026-08-04.

## 8. Engineering Manager Review

**Reviewer:** Engineering Manager (self-review trong cùng conversation, thay mặt vai trò EM theo yêu cầu STEP-2.1).

**Ngày:** 2026-08-04.

Checklist	Đánh giá
---	---

Scope Phase 2.2 có estimate hợp lý ( $\leq 2$ ngày Senior Dev)?	✓ Migration + 5 route + middleware + test — bám sát scope
Có dependency khoá tiến độ Phase 3 không?	✓ Bảng <code>users</code> từ Phase 2.2 là điều kiện tiên quyết Phase 3.1 — sequence đã đúng trong PLAN-MASTER
Rủi ro build native <code>bcrypt</code> trên Windows dev machine?	⚠ Đã ghi trong §5 "Rủi ro" — có fallback <code>bcryptjs</code> . EM chấp nhận.
Có blocker cho reviewer role trong Phase 2.3?	✓ Role <code>reviewer</code> định nghĩa đủ ở AD-A5, không cần chờ ADR thêm
Team có kỹ năng viết middleware Express + <code>bcrypt</code> /JWT?	✓ Senior Dev đã có kinh nghiệm; nếu vướng escalate Tech Lead

**Quyết định: APPROVE**, không thêm điều kiện ngoài 2 điều kiện của CTO.

Ký:

- **Engineering Manager: APPROVED**, 2026-08-04.

## 9. Referenced by

- `docs/plans/PLAN-labeling-studio-improve-2026-08-04/steps/STEP-2.2-auth-implementation.md` — bám ADR này để code.
- `docs/plans/PLAN-labeling-studio-improve-2026-08-04/steps/STEP-2.3-review-workflow.md` — dùng role model AD-A5.
- `docs/plans/PLAN-labeling-studio-improve-2026-08-04/steps/STEP-3.1-db-migration-schema.md` — FK `actor_id`, `completed_by` phụ thuộc `users.id`.
- `docs/plans/PLAN-labeling-studio-improve-2026-08-04/steps/STEP-3.5-image-done-status.md` — dùng `completed_by` FK.

## 10. Lịch sử

Ngày	Thay đổi	Người
---	---	---
2026-08-04	Tạo mới, self-run security-audit-stride, CTO+EM APPROVED (kèm 2 điều kiện #A1 #A2)	tech-lead + cto + em